Draw it or Lose it

CS 230 Project Software Design Template

Version 3.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 5

Document Revision History

Version	Date	Author	Comments
3.0	4/12/2026	Harsh Goswami	Last version of the design document for Draw it or Lose it.

Instructions
Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

The Gaming room currently runs a game called Draw It or Lose It and its android-only. The company is aiming to expand the game to a web-based platform and make a web application that is compatible with multiple operating systems and a variety of devices. This growth is going to need a scalable architecture that can support multiple users, teams, and game sessions while also making sure the web application can handle performance and is consistent across all platforms.

Requirements

Develop a web-based application using object-oriented programming principles with a standardized design pattern. The system should allow multiple teams and players to play the game while making sure there are unique names and identifiers for teams and the games. The design should be a singleton design pattern to make sure only one instance of the game is running at a time in memory for memory management and optimization. This should also manage game creation, player management, and team organization.

Design Constraints

Due to the system being distributed to a web-based application, there are a few constraints that dictate how the system will be built. One of the biggest ones is cross-platform compatibility. Making sure the game can run on multiple operating systems and devices is important, and using java and web-based technologies can help address this constraint. Another big important constraint is network communication and latency. Because the game is taking place online, we want to make sure there is proper communication between the clients and the servers so that they are efficient and reliable and without network delays or interruptions. The architect needs to be able to handle synchronization between players and also handle network instability.

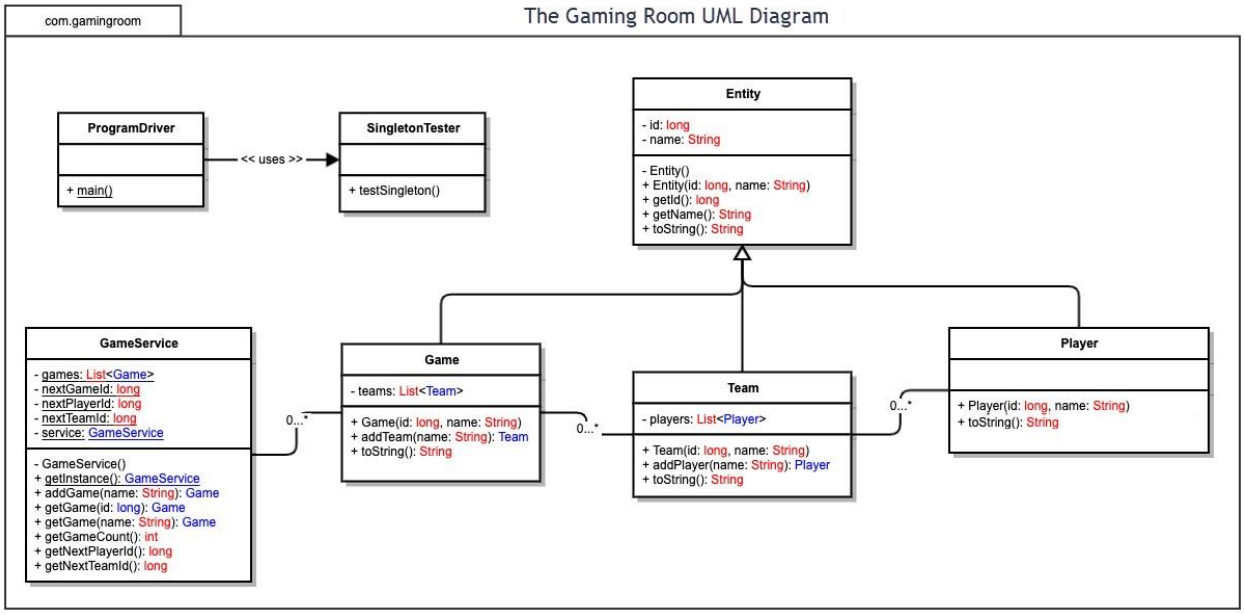
System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The UML class diagram below represents the domain model of the Draw It or Lose It application. We can easily see the major components of the system such as GameService, Game, Team, Player, Entity, ProgramDriver, and SingletonTester. At the core of all this we have Entity class which is the base class that has the common attribute shared by the other classes. These attributes also include a unique identifier called ID and a NAME. The classes such as Game, Team, and Player inherit from this entity class, and it demonstrates the object-oriented principle of inheritance. The GameService class manages all game related operations. It holds a list of game instances and generates a unique identifier for games, teams, and players. The Game class represents a single game instance and contains a list of teams participating in the game. A game at a given time can have multiple teams. The Team class also has a list of Player objects which establish another one-to-many relationship between Team and Player.

The Player class represents an individual participating within a team, since each player can only belong to exactly one team. The model demonstrates encapsulation by controlling how attributes such as identifiers and names are accessed through getter methods rather than direct access. This helps protect data integrity within the application. Lastly, the ProgramDriver class contains the main method that is responsible for running and executing the application.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Unix based OS that can support a Varity of different web server such as Apache and Java-based server application. Stable and secure but rarely used for large scale production because of its expensive nature and not exactly designed for large volume. Licensing costs are also higher.	Most common OS that's used for hosting web applications. Open-source platform with wide support and community with little to no licensing cost. It also supports server technologies like Apache, Nginx, and java applications. Has a good reputation for stable, secure and easy cloud scaling.	Supports enterprise web hosting and integrates well with Microsoft and its technologies such as .NET and IIS. Good reliability and also widely supported within the corporate environment. Downside to this is their licensing fees and higher system processing and resource usage compared to Linux.	Not suitable for hosting due to hardware and OS limitations. Designed for client usage rather than server processes. Lots of limitations such as battery, hardware, and more.
Client Side	Can still access the game through the browser like the rest because the game is running on a responsive web interface so the Mac user won't be required to download a separate application. We still want to make sure we are testing compatibility.	This user base is able to access this through browser such as Firefox or Chrome. Not a large userbase like Windows or Mobile but the application should still work but needs to make sure we test for compatibility.	Largest desktop user base so its critical there is compatibility for browsers such as Chrome, Edge, and Firefox. There needs to be a lot of browser testing to make sure the game performs consistently due to windows wide hardware variety.	Represents a huge portion on the client side. The website should be responsive to the design and in terms of functions, it needs to run properly on both IOS browsers and Android. The interface needs to auto adjust to the screen of the OS used. Cleaner touch-based interaction across all platforms.

Development Tools	There are a lot of professional development tools such as IntelliJ IDEA, Eclipse, Visual Studio Code, and more. While the tools are great, MacOS is generally more expensive and can actually cost more for development with added cost of Apple hardware.	Excellent for open-source development tools and its support. Linux integrates perfectly with containers like Docker and orchestration tools like Kubernetes. Since the OS is open sources, the development cost should be really low.	Like Linux, windows also supports a wide range of development tools and many enterprise development teams use Windows because of its compatibility with tools but the licensing cost for windows and the enterprise tools is also higher compare of Linux and definitely MacOS.	This platform primarily uses web development technologies like HTML5, CSS, and JavaScript for Browser-based applications. Developing applications on this platform is not idea but creating it for this OS is good.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**: The best operating platform would be Linux. Linux is open-source, reliable, secure, and highly scalable. It's a great option for hosting servers, and we could use it for server-side services. There are tons of support for Linux, and it also supports a wide variety of tools. Linux would also be a lost cost options compare to the rest since Linux doesn't require any sort of licensing like windows server or macOS.

2. **Operating Systems Architectures**: Linux supports modular architecture, virtualization, and containerization. These features allow scalable deployment and efficient resource use. The Linux kernel allows you to maange hardwaare reocuces and system processes which each application runs on their own user space. Draw it or lose it will have many players connected to it at the same time and Linux is perfect for that beuase its architecture supports multitasking and multi-user environment.

3. **Storage Management**: MySQL would be a great database management tool for storing game, team, and player data. MySQL works great with Linux and can support structured data storage such as game sessions, teams, user accounts, and players. For Darw it or lose it, MySQL would store player names, team memberships, and game session data without mixing up data and keeping it stored separately. Another advantage of MySQL is the backup system support, though data integrity, replication, and backup system feature it would hlep procet dat from loss or corruption.

4. **Memory Management**: Linux uses virtual memory and process isolation to efficiently allocate system resources, allowing multiple game sessions to run simultaneously. One of the key methods is virtual memeory. Virtual memeory is where the operating system usees both physical ram and disk space to manage running applications. Through this, it allows multiple game sessions and server processes to run at the same time without overloading the system's memory. Linux also has a process called process isolation which basically means that each process has its own space and processing power allocated to it. This helps with process interring with each other and helps with system stability.

5. **Distributed Systems and Networks**: The application should use a client-server model. Clients connect via HTTPS to a central server. Load balancing and cloud hosting can improve reliability and scalability. Network reliability must be considered when designing distributed systems. The application should include mechanisms for handling temporary connectivity issues, server outages, or latency. Cloud hosting and load balancing can help ensure the application remains available even when traffic increases or infrastructure failures occur.

6. **Security**: Use HTTPS encryption to protect data moving between clients and servers, secure authentication of secure login credentials, and passwords should be stored using hashing algorithms rather than plain text, and server-side access controls. Linux provides strong built-in security features to protect user data such as user permission management, firewall configuration, and even security modules that can help protect the system from unauthorized

access. In terms of overtime security, regular updates, patches, and monitoring tools are important to make sure the security system is up to date and reduce vulnerability.